# SEC: Introduction to Cloud Computing

Finding details of Museum Portraits
using Face Recognition on Amazon Web Services


**Name:** Himanshu Rai
**Roll No:** 1724135
**Course:** B.Sc. Mathematics (Hons)

**Project topic:** Cloud-Based Image Classification Application
**Submission Date:** 06 November 2026

Part A: What I built and how I built it

# 1 Introduction

Art Twin is a small website. You take or upload a selfie, and the page shows the three portrait paintings from a famous museum whose faces look most like yours. Behind the page, a computer program looks at faces, turns them into numbers, and compares those numbers.

The program does not run on my laptop. It runs on **Amazon Web Services (AWS)**, which is a collection of computers and tools that you rent over the internet instead of buying. This means anyone with the link can use the project from a phone, without installing anything.

This report explains, step by step and in simple words, what the project does, how face recognition works, which AWS services were used, and exactly how I set everything up. A second part compares running such programs on a normal CPU, on a GPU, and in the cloud.

# 2 The idea in simple words

## 2.1 How a computer "sees" a face

A photo is a grid of tiny coloured dots (pixels), and each dot is just three numbers for red, green and blue. A computer cannot "look" at a face the way we do, so we need a way to turn a face into something it can compare. The project does this in four steps.

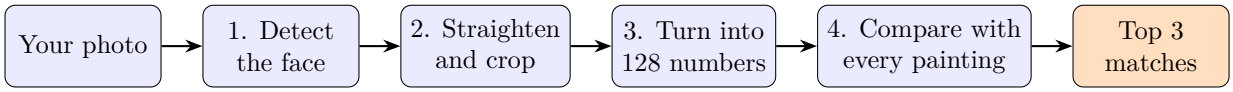



**Figure 1:** The four steps from a photo to the matching paintings.

1. **Detect.** A small neural network called **YuNet** finds where the face is and marks five points on it: both eyes, the nose tip and the two corners of the mouth.
2. **Straighten.** Using those five points, the face is rotated and resized into a standard 112×112 picture, so tilted or distant faces become comparable.
3. **Describe.** A second neural network called **SFace** reads the straightened face and produces a list of 128 numbers. This list is called an *embedding*. It is trained so that photos of the same person give similar lists, and different people give different lists.
4. **Compare.** The list for your face is compared with the stored list of every painting. The three most similar paintings are shown.

## 2.2 The mathematics of comparing

Each list of 128 numbers is a vector. Every vector is scaled to have length 1. For two such vectors $a$ and $b$, the similarity is their dot product, which equals the cosine of the angle between them:

$$\text{similarity}(a, b) = a \cdot b = \sum_{i=1}^{128} a_i b_i, \qquad -1 \le \text{similarity} \le 1.$$

The closer the number is to 1, the more alike the two faces are. If the paintings' vectors are stored as rows of a matrix $E$ and your vector is $f$, then the product $Ef$ gives the similarity with every painting at once. Sorting these values gives the top three.

### 2.3 Why not an ordinary classifier?

A normal image classifier chooses one label from a fixed list, such as “shoe” or “handbag”. There is no fixed list of faces, because every person is different. So instead of choosing a label, this project measures how close two faces are. It is still image recognition, but it works by matching rather than by choosing a category.

## 3 AWS services in simple terms

| Service | Everyday comparison | What it did here |
|---|---|---|
| EC2 | A computer you rent by the hour | My workshop. I used it to download the models, collect the paintings and prepare the data. |
| IAM | ID cards and permissions | Controlled who may do what in my account, including a role that let the workshop deploy the project. |
| S3 | A storage locker | Held the project package for a moment while it was being delivered. |
| Lambda | A worker who only turns up when called | Runs my Python code each time someone uses the page, so nothing is running (or billed) when nobody visits. |
| Function URL | The public front door address | Gives the project a web link that anyone can open. |

**Table 1:** The AWS services used.

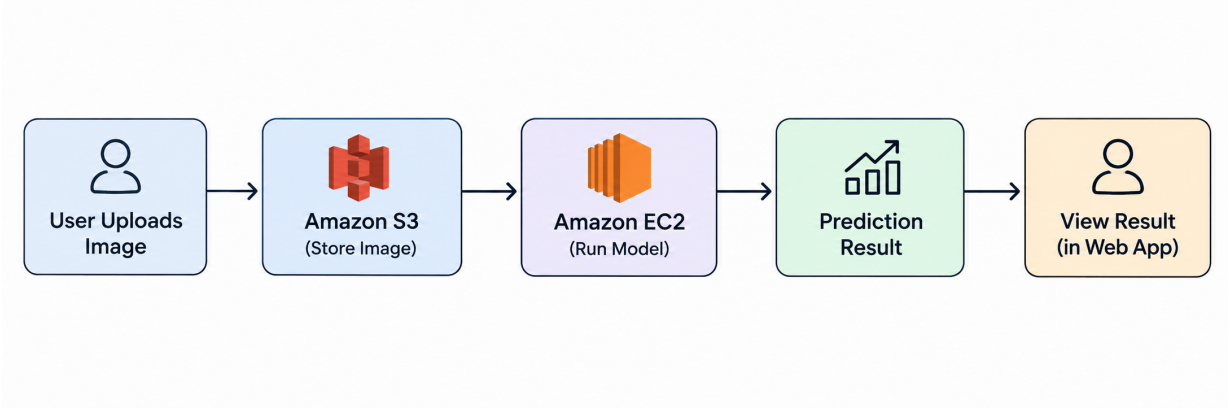



**Figure 2:** Architecture and workflow of the project on AWS: the data is prepared once on EC2 and then served by Lambda for every visitor.

## 4 How I built it, step by step

### Step 1: Securing the account

Before creating anything, I protected the account. I switched on multi-factor authentication for the main (root) user, created a separate administrator user for daily work, and set a budget alert so that I would be warned about any unexpected charges.

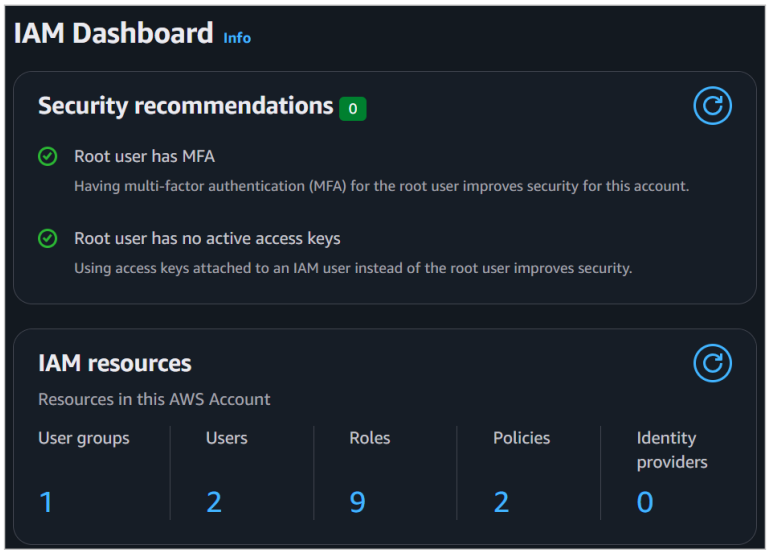



**Figure 3:** Multi-factor authentication and the IAM user.

## Step 2: Launching the EC2 computer

I launched a small Ubuntu Linux computer in the Mumbai region. I created a key pair (a private key file) for logging in, and a firewall rule that allows login only from my own internet address.

| Setting | Value |
|---|---|
| Region | Asia Pacific (Mumbai), ap-south-1 |
| Operating system | Ubuntu |
| Instance type | t3.micro (free-tier size) |
| Storage | 8 GB |
| Login | Key pair, SSH allowed from my IP only |

**Table 2:** EC2 settings.

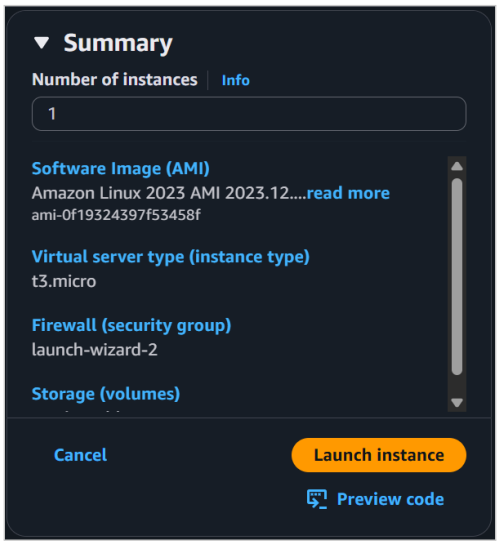



**Figure 4:** The launch summary of the EC2 instance.

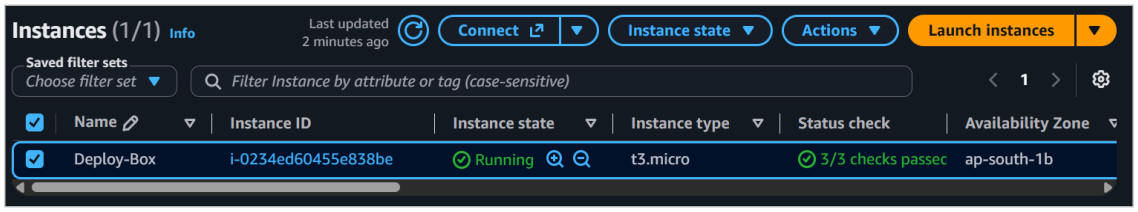



**Figure 5:** The instance in the running state.

## Step 3: Connecting to the computer

I connected to the computer from my own machine using SSH, a secure remote login. The key file proves that it is really me.

```
ssh -i "KeyEC2.pem" ubuntu@3.110.165.207
```

```
PS C:\Users\Himanshu Rai> cd downloads
PS C:\Users\Himanshu Rai\downloads> ssh -i "KeyEC2.pem" ubuntu@3.110.165.207
The authenticity of host '3.110.165.207 (3.110.165.207)' can't be established.
ED25519 key fingerprint is SHA256:XThzuJR7hGr4yxOm38FslZBqY/yehbnXsa9wW03wJng.
This host key is known by the following other names/addresses:
    C:\Users\Himanshu Rai/.ssh/known_hosts:6: 3.110.104.234
    C:\Users\Himanshu Rai/.ssh/known_hosts:12: 13.127.52.14
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '3.110.165.207' (ED25519) to the list of known hosts.
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-1014-aws x86_64)
```

**Figure 6:** Logged in to the EC2 computer.

## Step 4: Preparing the working environment

On the new computer I updated the system and created a separate Python environment for the project, so its libraries do not mix with anything else.

```
sudo apt update
sudo apt upgrade -y
sudo apt install python3-pip python3-venv -y

mkdir Art-Doppelganger-AWS
cd Art-Doppelganger-AWS

python3 -m venv venv
source venv/bin/activate
pip install opencv-python-headless numpy
```

```
Last login: Sun Oct 25 12:28:35 2026 from 171.76.236.246
ubuntu@ip-172-31-7-183:~$
```

**Figure 7:** The terminal after creating and activating the Python environment.

OpenCV is a well-known library for working with images. NumPy handles lists and tables of numbers quickly.

### Step 5: Downloading the two face models

The two neural networks are ready-made and free to use, published by the OpenCV project. I did not train them. YuNet is tiny (about 228 KB) and SFace is about 37 MB.

```
Z=https://github.com/opencv/opencv_zoo/raw/main/models

curl -sL -o yunet.onnx
$Z/face_detection_yunet/face_detection_yunet_2023mar.onnx

curl -sL -o sface.onnx
$Z/face_recognition_sface/face_recognition_sface_2021dec.onnx
```

```
ls -lh *.onnx
-rw-rw-r-- 1 ubuntu ubuntu  37M Nov  6 03:12 sface.onnx
-rw-rw-r-- 1 ubuntu ubuntu 228K Nov  6 03:12 yunet.onnx
```

**Figure 8:** The two downloaded model files.

### Step 6: Collecting the paintings

The paintings come from the Open Access collection of The Metropolitan Museum of Art in New York. It has a free public API (an address a program can ask for information), and its public-domain images may be used freely. A script I wrote searches for portrait paintings, downloads each image, finds the face in it, and saves that face's 128 numbers. Paintings with no clear face are skipped.

```python
import cv2, json, urllib.request, numpy as np

det = cv2.FaceDetectorYN.create("yunet.onnx", "", (320, 320), 0.7)
rec = cv2.FaceRecognizerSF.create("sface.onnx", "")

E = np.load("embeddings.npy"); M = json.load(open("meta.json", encoding="utf-8"))

def load(u):
  d = urllib.request.urlopen(
    urllib.request.Request(u,
          headers={"User-Agent": "Mozilla/5.0"},
          timeout=30).read()
  return cv2.imdecode(np.frombuffer(d, np.uint8), cv2.IMREAD_COLOR)

def embed(im):
    h, w = im.shape[:2]; det.setInputSize((w, h)); _, f = det.detect(im)
    if f is None:
        return None
    f = max(f, key=lambda a: a[2] * a[3]); v = rec.feature(rec.alignCrop(im, f)).flatten()
    return v / np.linalg.norm(v)

def rot(im, a):
    h, w = im.shape[:2]
    return cv2.warpAffine(im, cv2.getRotationMatrix2D((w / 2, h / 2), a, 1), (w, h))

def jpeg(im, q):
    return cv2.imdecode(cv2.imencode(".jpg", im, [cv2.IMWRITE_JPEG_QUALITY, q])[1], cv2.IMREAD_COLOR)

TESTS = {"original": lambda im: im,
         "blur": lambda im: cv2.GaussianBlur(im, (0, 0), 3),
         "darker 40%": lambda im: (im * 0.6).astype(np.uint8),
         "rotate 15 deg": lambda im: rot(im, 15),
         "half size": lambda im: cv2.resize(im, None, fx=.5, fy=.5),
         "JPEG quality 10": lambda im: jpeg(im, 10)}


S = {k: [0, 0, 0] for k in TESTS}

for i, m in enumerate(M):
    try:
        im = load(m["image"])
    except Exception:
        continue
    if im is None:
        continue
    for name, fn in TESTS.items():
        v = embed(fn(im))
        if v is None:
            continue
        order = np.argsort(E @ v)[::-1]
        S[name][0] += int(order[0] == i)
    S[name][1] += int(i in order[:3])
    S[name][2] += 1
print(f"{'condition':18s}{'top-1':>8s}{'top-3':>8s}{'images':>8s}")

for k, (a, b, n) in S.items():
    print(f"{k:18s}{(100*a/n if n else 0):7.1f}%{(100*b/n if n else 0):7.1f}%{n:8d}")
```

The script saves two files: `embeddings.npy` (one row of 128 numbers per painting) and `meta.json` (title, artist, date, image link and museum link for each painting).

```
ubuntu@ip-172-31-7-183:~$ tree -L 2
.
├── Art-Doppelganger-AWS
│   └── art-twin
├── art-twin-venv
│   ├── bin
│   ├── include
│   ├── lib
│   ├── lib64 -> lib
│   └── pyvenv.cfg
├── aws
│   ├── README.md
│   ├── THIRD_PARTY_LICENSES
│   ├── dist
│   └── install
├── awscliv2.zip
├── sface.onnx
├── v
│   ├── bin
│   ├── include
│   ├── lib
│   ├── lib64 -> lib
│   └── pyvenv.cfg
└── yunet.onnx

15 directories, 8 files
```

**Figure 9:** The dataset script finishing, with the files it produced.

### Step 7: The matching code

This is the core of the project. It reads the uploaded photo, finds the face, produces its 128 numbers and compares them with all the paintings in one line.

```
face = max(faces, key=lambda f: f[2] * f[3])
feat = rec.feature(rec.alignCrop(img, face)).flatten()
feat = feat / np.linalg.norm(feat)
sims = E @ feat                    # similarity with every painting
top = np.argsort(sims)[::-1][:3]   # indexes of the 3 best
```

The scores this gives are relative. A selfie compared with a painting of a different person normally scores low, so the project shows the *closest* faces, not proof of identity.

### Step 8: The web page

The page is a single file written in HTML, CSS and JavaScript. The visitor chooses a photo or uses the camera. The browser shrinks the picture and sends it to the program, then displays the user's photo beside the best match and two more matches below. Photos are processed in memory and are never saved.

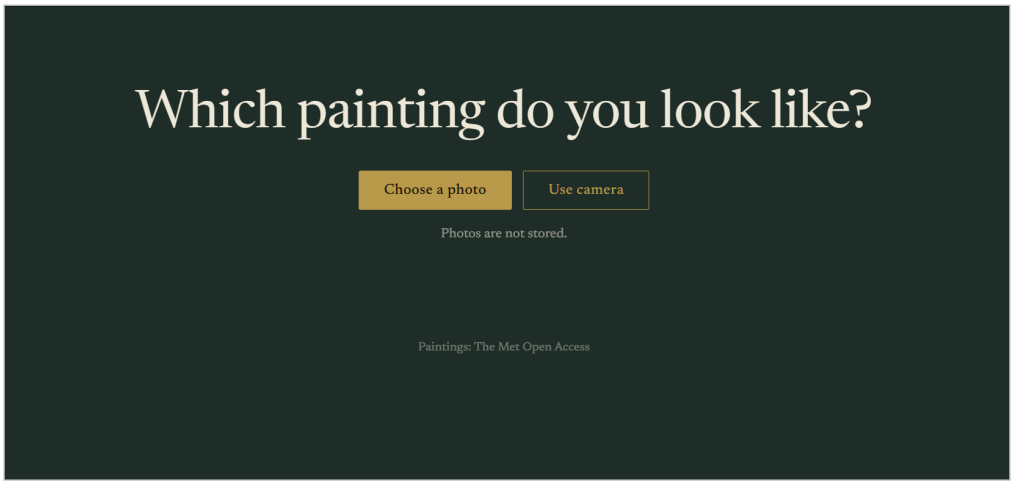



**Figure 10:** The home page.

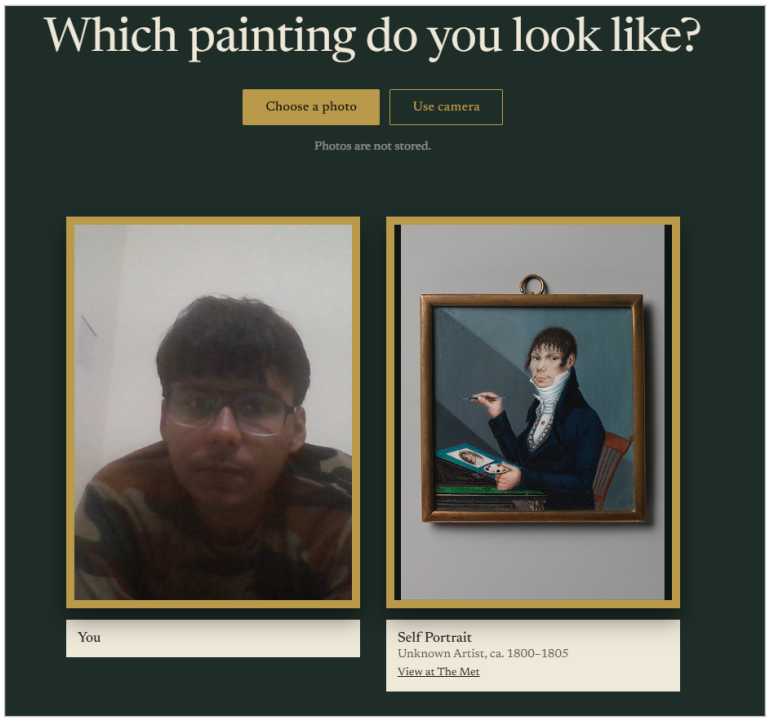



**Figure 11:** A result: the user's photo beside the matched painting.

### Step 9: Putting it online

A script packages the program, the two models and the data into one zip file, uploads it through S3, and creates a Lambda function with a public address. Lambda needs the libraries built for its own Linux system, which the script handles. The same address that shows the page also receives the photos.

```
pip install --target build --platform manylinux2014_x86_64
\--only-binary=:all: --python-version 3.12
\"opencv-python-headless==4.10.0.84" "numpy<2"

zip -r function.zip build/ # code + models + data

aws s3 cp function.zip s3://<temporary-bucket>/
```

AWS CloudShell, the browser terminal, was unavailable at first because my new account was still being verified. So I used the EC2 computer as the place to run these commands, with an IAM role giving it permission instead of storing passwords on it.

```
ubuntu@ip-172-31-7-183:~/Art-Doppelganger-AWS/art-twin$ bash deploy.sh
upload: ./function.zip to s3://art-twin-445617516690-tmp/function.zip
(concurrency cap not set, fine for a class demo)

LIVE AT:
https://flihx7v6sq3pp4j62rd6wnwwam0fbccz.lambda-url.ap-south-1.on.aws/
```

**Figure 12:** The deployment finishing with the public address.

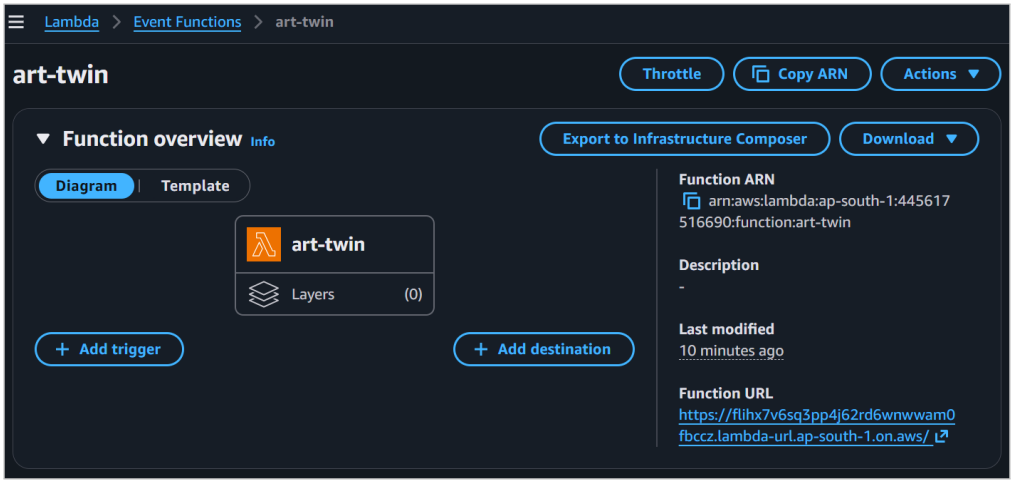



**Figure 13:** The Lambda function in the AWS console.

## Step 10: Testing and cost control

I tested with several clear front-facing photos and checked that the page loads, matches appear, and the camera option works. Finally, I shut down what I no longer needed (EC2 instance, public IP addresses and the temporary bucket) and checked the billing page, because forgotten resources keep charging.

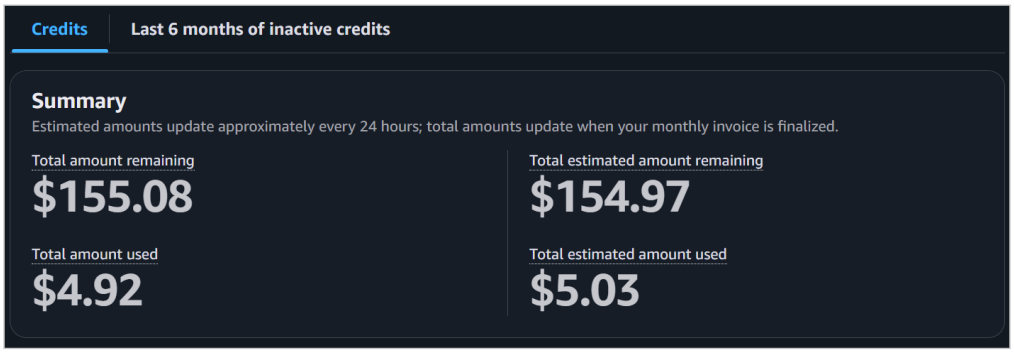



**Figure 14:** Billing page after clean-up.

# 5 Problems I faced and how I solved them

| Problem | What I did |
|---|---|
| CloudShell was blocked (account under verification) | Used the EC2 computer with an IAM role as the deployment machine. |
| OpenCV could not be imported | Created a fresh Python environment and reinstalled the libraries. |
| Python 3.12 was not available on the EC2 system | Built the data with the available Python version and packaged the libraries for Lambda's own Python 3.12. |
| Many museum API requests failed | Changed the requests, skipped bad records and added retries. |
| The package was close to Lambda's size limit | Pinned an older, smaller OpenCV and removed unneeded files. |
| Deploy script errors (unsupported option, bucket already existed) | Removed the option and made the script continue if the bucket existed. |

**Table 3:** Problems and fixes.

# 6 Limitations and responsible use

- The set of paintings is small, so the best match is sometimes just the least bad one.
- Painted faces look different from photographs, so the matches are for fun and are not scientific proof of resemblance.
- Results depend on light, angle and photo quality.
- The public link can be used by anyone, so it should be switched off when not needed.
- Face data is personal. Photos are not stored, and any real use such as attendance or ID checks would need consent, a stronger model and a check that the person is really present.

# What comes next

Part B of this report compares running image-recognition programs on a normal CPU, on a GPU and in the cloud, with measurements.